MODULE 1 · PRE-READ

Module 1, Pre-Read

Onboard Claude to Your Product** · superpower:
“Origin Story”

Claude Code for PMs · Pre-Read · Module 1

How to have Claude Code persistently know you

15 min. Skim before the live session.

You do not write or read any code in this course. You work inside your own repo. Finish the three steps on the Pre-Class Setup page before Module 1.

Where we are

You're the new PM at Rook Industries, which builds software for actual superheroes. Your product is Rook Dispatch, which gets a superhero to where they're needed when there's an emergency. It works out who is close enough and free enough to help, and gets hold of them. Release 4.2 shipped on August 12, shortly before you arrived, and it has problems.

Where things stand. You just joined. Release 4.2 shipped twelve days ago, before you started. That's everything anyone's told you so far.

Before the first session

- **Claude Pro or higher.** Claude Code is not in the free tier.
- A free **personal GitHub** account at github.com/signup, not your employer's.
- The **Claude app** from claude.com/download. On Windows, also install Git for Windows.
- Finish the three steps on the Pre-Class Setup page before Day 1. Open a session on your `claude-code-for-pms-final` folder and type `check my setup`. Checks 4 and 5 confirm the rook-wiki and rook-database connectors. At the end of each day, type `wrap up` so your files are saved to your repo for the next day.

Keep that repo **Public** so your instructor can see your work.

What you'll do in the session

1 hands-on lab, 40 minutes of lab time in total.

1. **Build a context file** (40 min), You're going to give Claude everything Rook knows about itself, and then ask it something you couldn't have asked before.

What lands in your repo

`01-origin-story/prompts.md`, the prompts you wrote yourself. Claude Code writes the file at the end of the session; you never open it by hand. Commit it before you close.

Read a hundred things at once, understand them, and find where they disagree

15 min. Skim before the live session.

You do not write or read any code in this course. You work inside your own repo, created from the course template in Module 1.

Where we are

You're the new PM at Rook Industries, which builds software for actual superheroes. Your product is Rook Dispatch, which gets a superhero to where they're needed when there's an emergency. It works out who is close enough and free enough to help, and gets hold of them. Release 4.2 shipped on August 12, shortly before you arrived, and it has problems.

Where things stand. Last session you built a context file and met the company, but you still don't know what actually went wrong with release 4.2. Today two new piles of raw customer data land on your desk.

The technique this module turns on

Don't ask for a summary

- **What you've probably tried:** `Summarize these.` Gets you a paragraph that sounds right and hides everything. You can't check it, and you can't act on it.
- **What works instead:** `Group these, tell me how many are in each group, and quote me one line from each.` Gets you something you can act on, and something you can check against the actual documents.

Why this matters for Claude Code. This isn't just a better prompt, but something only Claude Code can actually deliver. It doesn't answer from a chat's memory of what you pasted in. It reads Customer interviews on rook-wiki and support_tickets on rook-database every time, so a count and a quote trace straight back to a source you can reopen. A summary from a chat hides where it came from, while a count from Claude Code doesn't have to.

What you'll do in the session

3 hands-on labs, 73 minutes of lab time in total.

1. **Lab A: Point Claude at the Interviews** (28 min), Three steps: point Claude at a room full of interviews, push past what's obvious, then commit to a read of your own.
2. **Lab B: Point Claude at the Tickets** (20 min), Same technique, a very different pile: point Claude at the tickets, push past the obvious, then flag what changed after the

date that matters.

3. **Lab C: Point Claude at Both Piles** (25 min), Three steps: bring the two reads together, push it further, then check it against what you posted in Lab A.

What lands in your repo

`02-super-hearing/prompts.md`, the prompts you wrote yourself. Claude Code writes the file at the end of the session; you never open it by hand. Commit it before you close.

MODULE 1 · PRE-READ · CLAUDE CODE FOR PMS

Keep this open while you build.

Claude Code for PMs · Reference companion